arXiv:2309.03199v2 [eess.AS] 9 Jan 2024

MATCHA-TTS: A FAST TTS ARCHITECTURE WITH CONDITIONAL FLOW MATCHING

Shivam Mehta, Ruibo Tu, Jonas Beskow, Éva Székely, Gustav Eje Henter

Division of Speech, Music and Hearing, KTH Royal Institute of Technology, Stockholm, Sweden

ABSTRACT

We introduce Matcha-TTS, a new encoder-decoder architecture for speedy TTS acoustic modelling, trained using optimal-transport conditional flow matching (OT-CFM). This yields an ODE-based decoder capable of high output quality in fewer synthesis steps than models trained using score matching. Careful design choices additionally ensure each synthesis step is fast to run. The method is probabilistic, non-autoregressive, and learns to speak from scratch without external alignments. Compared to strong pre-trained baseline models, the Matcha-TTS system has the smallest memory footprint, rivals the speed of the fastest model on long utterances, and attains the highest mean opinion score in a listening test.

***Index Terms*—** Diffusion models, flow matching, speech synthesis, text-to-speech, acoustic modelling

1. INTRODUCTION

Diffusion probabilistic models (DPMs) (cf. [1]) are currently setting new standards in deep generative modelling on continuous-valued data-generation tasks such as image synthesis [2, 3], motion synthesis [4, 5], and speech synthesis [6, 7, 8, 9, 10] – the topic of this paper. DPMs define a diffusion process which transforms the *data* (a.k.a. *target*) distribution to a *prior* (a.k.a. *source*) distribution, e.g., a Gaussian. They then learn a *sampling process* that reverses the diffusion process. The two processes can be formulated as forward- and reverse-time stochastic differential equations (SDEs) [11]. Solving a reverse-time SDE initial value problem generates samples from the learnt data distribution. Furthermore, each reverse-time SDE has a corresponding ordinary differential equation (ODE), called the *probability flow ODE* [11, 12], which describes (and samples from) the exact same distribution as the SDE. The probability flow ODE is a deterministic process for turning source samples into data samples, similar to continuous-time normalising flows (CNF) [13], but without the need to backpropagate through expensive ODE solvers or approximate the reverse ODE using adjoint variables [13].

The SDE formulation of DPMs is trained by approximating the score function (the gradients of the log probability density) of the data distribution [11]. The training objective takes the form of a mean squared error (MSE) which can be derived from an evidence lower bound (ELBO) on the likelihood. This is fast and simple and, unlike typical normalising flow models, does not impose any restrictions on model architecture. But whilst they allow efficient training without numerical SDE/ODE solvers, DPMs suffer from slow synthesis speed, since each sample requires numerous iterations (steps), computed in sequence, to accurately solve the SDE. Each such step requires that an entire neural network be evaluated. This slow synthesis speed has long been the main practical issue with DPMs.

This paper introduces *Matcha-TTS*[1], a probabilistic and non-autoregressive, fast-to-sample-from TTS acoustic model based on continuous normalising flows. There are two main innovations:

1. To begin with, we propose an improved encoder-decoder TTS architecture that uses a combination of 1D CNNs and Transformers in the decoder. This reduces memory consumption and is fast to evaluate, improving synthesis speed.
2. Second, we train these models using optimal-transport conditional flow matching (OT-CFM) [14], which is a new method to learn ODEs that sample from a data distribution. Compared to conventional CNFs and score-matching probability flow ODEs, OT-CFM defines simpler paths from source to target, enabling accurate synthesis in fewer steps than DPMs.

Experimental results demonstrate that both innovations accelerate synthesis, reducing the trade-off between speed and synthesis quality. Despite being fast and lightweight, Matcha-TTS learns to speak and align without requiring an external aligner. Compared to strong pre-trained baseline models, Matcha-TTS achieves fast synthesis with better naturalness ratings. Audio examples and code are provided at https://shivammehta25.github.io/Matcha-TTS/.

2. BACKGROUND

2.1. Recent encoder-decoder TTS architectures

DPMs have been applied to numerous speech-synthesis tasks with impressive results, including waveform generation [6, 10] and end-to-end TTS [7]. Diff-TTS [9] was first to apply DPMs for acoustic modelling. Shortly after, Grad-TTS [8] conceptualised the diffusion process as an SDE. Although these models, and descendants like Fast Grad-TTS [15], are non-autoregressive, TorToiSe [16] demonstrated DPMs in an autoregressive TTS model with quantised latents.

The above models – like many modern TTS acoustic models – use an encoder-decoder architecture with Transformer blocks in the encoder. Many models, e.g., FastSpeech 1 and 2 [17, 18], use sinusoidal position embeddings for positional dependences. This has been found to generalise poorly to long sequences; cf. [19]. Glow-TTS [20], VITS [21], and Grad-TTS instead use relative positional embeddings [22]. Unfortunately, these treat inputs outside a short context window as a "bag of words", often resulting in unnatural prosody. LinearSpeech [23] instead employed rotational position embeddings (RoPE) [24], which have computational and memory advantages over relative embeddings and generalise to longer distances [25, 19]. Matcha-TTS thus uses Transformers with RoPE in the encoder, reducing RAM use compared to Grad-TTS. We believe ours is the first SDE or ODE-based TTS method to use RoPE.

This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation and by the Industrial Strategic Technology Development Program (grant no. 20023495) funded by MOTIE, Korea.

[1] We call our approach Matcha-TTS because it uses flow matching for TTS, and because the name sounds similar to "matcha tea", which some people prefer over Taco(tron)s.

Modern TTS architectures also differ in terms of decoder network design. The normalising-flow based methods Glow-TTS [20] and OverFlow [26] use dilated 1D-convolutions. DPM-based methods like [9, 27] likewise use 1D convolutions to synthesise mel spectrograms. Grad-TTS [8], in contrast, uses a U-Net with 2D-convolutions. This treats mel spectrograms as images and implicitly assumes translation invariance in both time and frequency. However, speech mel-spectra are not fully translation-invariant along the frequency axis, and 2D decoders generally require more memory as they introduce an extra dimension to the tensors. Meanwhile, non-probabilistic models like FastSpeech 1 and 2 have demonstrated that decoders with (1D) Transformers can learn long-range dependencies and fast, parallel synthesis. Matcha-TTS also uses Transformers in the decoder, but in a 1D U-Net design inspired by the 2D U-Nets in the Stable Diffusion image-generation model [3].

Whilst some TTS systems, e.g., FastSpeech [17], rely on externally-supplied alignments, most systems are capable of learning to speak and align at the same time, although it has been found to be important to encourage or enforce monotonic alignments [28, 29] for fast and effective training. One mechanism for this is monotonic alignment search (MAS), used by, e.g., Glow-TTS [20] and VITS [21]. Grad-TTS [8], in particular, uses a MAS-based mechanism which they term *prior loss* to quickly learn to align input symbols with output frames. These alignments are also used to train a deterministic duration predictor minimising MSE in the log domain. Matcha-TTS uses these same methods for alignment and duration modelling. Finally, Matcha-TTS differs by using *snake beta* activations from BigVGAN [30] in all decoder feedforward layers.

2.2. Flow matching and TTS

Currently, some of the highest-quality TTS systems either utilise DPMs [8, 16] or discrete-time normalising flows [21, 26], with continuous-time flows being less explored. Lipman et al. [14] recently introduced a framework for synthesis using ODEs that unifies and extends probability flow ODEs and CNFs. They were then able to present an efficient approach to learn ODEs for synthesis, using a simple vector-field regression loss called *conditional flow matching* (CFM), as an alternative to learning score functions for DPMs or using numerical ODE solvers at training time like classic CNFs [13]. Crucially, by leveraging ideas from optimal transport, CFM can be set up to yield ODEs that have simple vector fields that change little during the process of mapping samples from the source distribution onto the data distribution, since it essentially just transports probability mass along straight lines. This technique is called *OT-CFM*; *rectified flows* [31] represent concurrent work with a similar idea. The simple paths mean that the ODE can be solved accurately using few discretisation steps, i.e., accurate model samples can be drawn with fewer neural-network evaluations than DPMs, enabling much faster synthesis for the same quality.

CFM is a new technique that differs from earlier approaches to speed up SDE/ODE-based TTS, which most often were based on distillation (e.g., [27, 15, 32]). Prior to Matcha-TTS, the only public preprint on CFM-based acoustic modelling was the Voicebox model from Meta [33]. Voicebox (VB) is a system that performs various text-guided speech-infilling tasks based on large-scale training data, with its English variant (VB-En) being trained on 60k hours of proprietary data. VB differs substantially from Matcha-TTS: VB performs TTS, denoising, and text-guided acoustic infilling trained using a combination of masking and CFM, whereas Matcha-TTS is a pure TTS model trained solely using OT-CFM. VB uses convolutional positional encoding with AliBi [19] self-attention bias, whilst our text encoder uses RoPE. In contrast to VB, we train on standard data and make code and checkpoints publicly available. VB-En consumes 330M parameters, which is 18 times larger than the Matcha-TTS model in our experiments. Also, VB uses external alignments for training whereas Matcha-TTS learns to speak without them.

3. METHOD

We now outline flow-matching training (in Sec. 3.1) and then (in Sec. 3.2) give details on our proposed TTS architecture.

3.1. Optimal-transport conditional flow matching

We here give a high-level overview of flow matching, first introducing the probability-density path generated by a vector field and then leading into the OT-CFM objective used in our proposed method. Notation and definitions mainly follow [14].

Let $\boldsymbol{x}$ denote an observation in the data space $\mathbb{R}^d$, sampled from a complicated, unknown data distribution $q(\boldsymbol{x})$. A *probability density path* is a time-dependent probability density function, $p_t : [0,1] \times \mathbb{R}^d \to \mathbb{R} > 0$. One way to generate samples from the data distribution q is to construct a probability density path p_t, where $t \in [0,1]$ and $p_0(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}; \mathbf{0}, \boldsymbol{I})$ is a prior distribution, such that $p_1(\boldsymbol{x})$ approximates the data distribution $q(\boldsymbol{x})$. For example, CNFs first define a vector field $\boldsymbol{v}_t : [0,1] \times \mathbb{R}^d \to \mathbb{R}^d$, which generates the flow $\phi_t : [0,1] \times \mathbb{R}^d \to \mathbb{R}^d$ through the ODE

$$\tfrac{d}{dt}\phi_t(\boldsymbol{x}) = \boldsymbol{v}_t(\phi_t(\boldsymbol{x})); \qquad \phi_0(\boldsymbol{x}) = \boldsymbol{x}. \tag{1}$$

This generates the path p_t as the marginal probability distribution of the data points. We can sample from the approximated data distribution p_1 by solving the initial value problem in Eq. (1).

Suppose there exists a known vector field $\boldsymbol{u}_t$ that generates a probability path p_t from p_0 to $p_1 \approx q$. The flow matching loss is

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{t,p_t(\boldsymbol{x})}\|\boldsymbol{u}_t(\boldsymbol{x}) - \boldsymbol{v}_t(\boldsymbol{x};\theta)\|^2, \tag{2}$$

where $t \sim \mathbb{U}[0,1]$ and $\boldsymbol{v}_t(\boldsymbol{x};\theta)$ is a neural network with parameters θ. Nevertheless, flow matching is intractable in practice because it is non-trivial to get access to the vector field $\boldsymbol{u}_t$ and the target probability p_t. Therefore, conditional flow matching instead considers

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t,q(\boldsymbol{x}_1),p_t(\boldsymbol{x}|\boldsymbol{x}_1)}\|\boldsymbol{u}_t(\boldsymbol{x}|\boldsymbol{x}_1) - \boldsymbol{v}_t(\boldsymbol{x};\theta)\|^2. \tag{3}$$

This replaces the intractable marginal probability densities and the vector field with conditional probability densities and conditional vector fields. Crucially, these are in general tractable and have closed-form solutions, and one can furthermore show that $\mathcal{L}_{\mathrm{CFM}}(\theta)$ and $\mathcal{L}_{\mathrm{FM}}(\theta)$ both have identical gradients with respect to θ [14].

Matcha-TTS is trained using optimal-transport conditional flow matching (OT-CFM) [14], which is a CFM variant with particularly simple gradients. The OT-CFM loss function can be written

$$\mathcal{L}(\theta) = \mathbb{E}_{t,q(\boldsymbol{x}_1),p_0(\boldsymbol{x}_0)}\|\boldsymbol{u}_t^{\mathrm{OT}}(\phi_t^{\mathrm{OT}}(\boldsymbol{x})|\boldsymbol{x}_1) - \boldsymbol{v}_t(\phi_t^{\mathrm{OT}}(\boldsymbol{x})|\boldsymbol{\mu};\theta)\|^2, \tag{4}$$

defining $\phi_t^{\mathrm{OT}}(\boldsymbol{x}) = (1-(1-\sigma_{\min})t)\boldsymbol{x}_0 + t\boldsymbol{x}_1$ as the flow from $\boldsymbol{x}_0$ to $\boldsymbol{x}_1$ where each datum $\boldsymbol{x}_1$ is matched to a random sample $\boldsymbol{x}_0 \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ as in [14]. Its gradient vector field – whose expected value is the target for the learning – is then $\boldsymbol{u}_t^{\mathrm{OT}}(\phi_t^{\mathrm{OT}}(\boldsymbol{x}_0)|\boldsymbol{x}_1) = \boldsymbol{x}_1 - (1-\sigma_{\min})\boldsymbol{x}_0$, which is linear, time-invariant, and only depends on $\boldsymbol{x}_0$ and $\boldsymbol{x}_1$. These properties enable easier and faster training, faster generation, and better performance compared to DPMs.

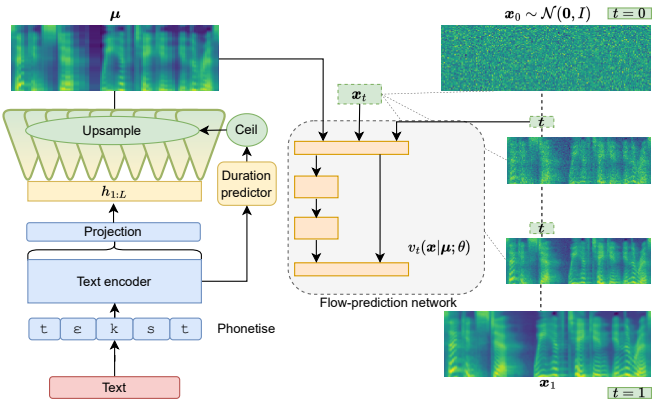

Fig. 1: Overview of the proposed approach at synthesis time.

In the case of Matcha-TTS, $\boldsymbol{x}_1$ are acoustic frames and $\boldsymbol{\mu}$ are the conditional mean values of those frames, predicted from text using the architecture described in the next section. σ_{min} is a hyperparameter with a small value (`1e-4` in our experiments).

3.2. Proposed architecture

Matcha-TTS is a non-autoregressive encoder-decoder architecture for neural TTS. An overview of the architecture is provided in Fig. 1. Text encoder and duration predictor architectures follow [20, 8], but use rotational position embeddings [24] instead of relative ones. Alignment and duration-model training follow use MAS and the prior loss $\mathcal{L}_{\text{enc}}$ as described in [8]. The predicted durations, rounded up, are used to upsample (duplicate) the vectors output by the encoder to obtain $\boldsymbol{\mu}$, the predicted average acoustic features (e.g., mel-spectrogram) given the text and the chosen durations. This mean is used to condition the decoder that predicts the vector field $\boldsymbol{v}_t(\phi_t^{\text{OT}}(\boldsymbol{x}_0)|\boldsymbol{\mu};\theta)$ used for synthesis, but is not used as the mean for the initial noise samples $\boldsymbol{x}_0$ (unlike Grad-TTS).

Fig. 2 shows the Matcha-TTS decoder architecture. Inspired by [3], it is a U-Net containing 1D convolutional residual blocks to downsample and upsample the inputs, with the flow-matching step $t \in [0, 1]$ embedded as in [8]. Each residual block is followed by a Transformer block, whose feedforward nets use snake beta activations [30]. These Transformers do not use any position embeddings, since between-phone positional information already has been baked in by the encoder, and the convolution and downsampling operations act to interpolate these between frames within the same phone and distinguish their relative positions from each other. This decoder network is significantly faster to evaluate and consumes less memory than the 2D convolutional-only U-Net used by Grad-TTS [8].

4. EXPERIMENTS

To validate the proposed approach we compared it to three pre-trained baselines in several experiments, including a listening test. All experiments were performed on NVIDIA RTX 3090 GPUs. See shivammehta25.github.io/Matcha-TTS/ for audio and code.

4.1. Data and systems

We performed our experiments on the standard split of the LJ Speech dataset[2] (a female US English native speaker reading public-domain

[2] https://keithito.com/LJ-Speech-Dataset/

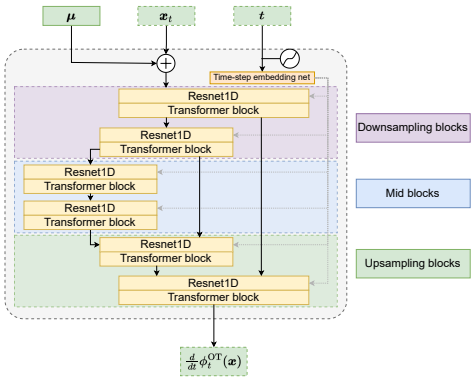

Fig. 2: Matcha-TTS decoder (the flow-prediction network in Fig. 1).

texts), training a version of the Matcha-TTS architecture on this data. We used the same encoder and duration predictor (i.e., the same hyperparameters) as [8], just different position embeddings in the encoder. Our trained flow-prediction network (decoder) used two downsampling blocks, followed by two midblocks and two upsampling blocks, as shown in Fig. 2. Each block had one Transformer layer with hidden dimensionality 256, 2 heads, attention dimensionality 64, and 'snakebeta' activations [30]. Phonemizer[3] [34] with the `espeak-ng` backend was used to convert input graphemes to IPA phones. We trained for 500k updates on 2 GPUs with batch size 32 and learning rate `1e-4`, labelling our trained system **MAT**.

MAT was compared to three widely used neural TTS baseline approaches with pre-trained checkpoints available for LJ Speech, namely Grad-TTS[4] [8] (label **GRAD**), a strong DPM-based acoustic model, FastSpeech 2 (**FS2**), a fast non-probabilistic acoustic model, and **VITS**[5], a strong probabilistic end-to-end TTS system with discrete-time normalising flows. The baselines used the official checkpoints from the respective linked repositories. For FS2, which does not provide an official implementation, we instead used the checkpoint from Meta's FairSeq[6]. To decouple the effects of CFM training from those due to the new architecture, we also trained the GRAD architecture using the OT-CFM objective instead, using the same optimiser hyperparameters as for MAT. This produced the ablation labelled **GCFM**. For all acoustic models (i.e., all systems except VITS), we used the pre-trained HiFi-GAN [35] LJ Speech checkpoint `LJ_V1`[7] for waveform generation, with a denoising filter as introduced in [36] at a strength of `2.5e-4`. As a top line, our experiments also included vocoded held-out speech, labelled **VOC**.

ODE-based models, e.g., DPMs, allow trading off speed against quality. We therefore evaluated synthesis from the trained ODE-based systems with a different number of steps for the ODE solver. Like [8], we used the first-order Euler forward ODE-solver, where the number of steps is equal to the number of function (i.e., neural-network) evaluations, commonly abbreviated *NFE*. This gave rise to multiple *conditions* for some systems. We labelled these conditions **MAT-n**, **GRAD-n**, and **GCFM-n**, n being the NFE. We used NFE 10 or less, since [8] reported that NFE 10 and 100 gave the same MOS for Grad-TTS (NFE 50 is the official code default). All conditions used a temperature of 0.667 for synthesis, similar to [8]. Table 1 provides an overview of the conditions in the evaluation.

[3] https://github.com/bootphon/phonemizer
[4] https://github.com/huawei-noah/Speech-Backbones/tree/main/Grad-TTS
[5] https://github.com/jaywalnut310/vits
[6] https://github.com/facebookresearch/fairseq
[7] https://github.com/jik876/hifi-gan/

Condition	Params.	RAM	RTF ($\mu\pm\sigma$)	WER	MOS
VOC	13.9M	-	0.001±0.001	1.97	4.13±0.09
FS2	41.2M	6.0	**0.010**±0.004	4.18	3.29±0.09
VITS	36.3M	12.4	0.074±0.083	2.52	3.71±0.08
GRAD-10	**14.8M**	7.8	0.049±0.013	3.44	3.49±0.08
GRAD-4	"	"	0.019±0.006	3.69	3.20±0.09
GCFM-4	"	"	0.019±0.004	2.70	3.57±0.08
MAT-10	18.2M	**4.8**	0.038±0.019	**2.09**	**3.84**±0.08
MAT-4	"	"	0.019±0.008	2.15	3.77±0.07
MAT-2	"	"	0.015±0.006	2.34	3.65±0.08

Table 1: Conditions in the evaluation (with the NFE for ODE-based methods) and their number of parameters, minimum GPU RAM needed to train (GiB), real-time factor (including vocoding time) on the test set, ASR WER in percent, and mean opinion score with 95%-confidence interval. The best TTS condition in each column is bold. The parameter count and RTF for VOC pertain to the vocoder.

4.2. Evaluations, results, and discussion

We evaluated our approach both objectively and subjectively. First we measured parameter count and maximum memory use during training (at batch size 32 and fp16) of all systems, with results listed in Table 1. We see that MAT is approximately the same size as GRAD/GCFM, and smaller than all other systems. In particular, it is smaller than VITS also after adding the vocoder (13.9M parameters) to the MAT parameter count. More importantly, it uses less memory than all baselines, which (more than parameter count) is the main limiter on how large and powerful models that can be trained.

After training the systems, we assessed the synthesis speed and intelligibility of the different conditions, by computing the real time factor (RTF) mean and standard deviation when synthesising the test set, and evaluating the word error rate (WER) when applying the Whisper `medium` [37] ASR system to the results, since the WERs of strong ASR systems correlate well with intelligibility [38]. The results, in Table 1, suggest that MAT is the most intelligible system, even using only two synthesis steps. MAT is also much faster than VITS, equally fast or faster than GRAD/GCFM at the same NFE, and only slightly slower than FS2 when at the fastest setting.

To evaluate the naturalness of the synthesised audio we ran a mean opinion score (MOS) listening test. We selected 40 utterances (4 groups of 10) of different lengths from the test set and synthesised each utterance using all conditions, loudness-normalising every stimulus using EBU R128. 80 subjects (self-reported as native English speakers using headphones) were crowdsourced through Prolific to listen to and rate these stimuli. For each stimulus, listeners were asked "How natural does the synthesised speech sound?", and provided responses on an integer rating scale from 1 ("Completely unnatural") to 5 ("Completely natural") adopted from the Blizzard Challenge [39]. Each group of 10 utterances was evaluated by 20 listeners, who were paid £3 for a median completion time of 13 mins. Inattentive listeners were filtered out and replaced in exactly the same way as in [26]. In the end we obtained 800 ratings for each condition. The resulting MOS values, along with confidence intervals based on a normal approximation, are listed in Table 1. We note that, since MOS values depend on many variables external to stimulus quality, e.g., listener demographics and instructions (see [40, 41]), they should not be treated as an absolute metric. Comparing our MOS values to other papers is thus unlikely to be meaningful.

Applying t-tests to all pairs of conditions, all differences were found to be statistically significant at the $\alpha = 0.05$ level except the pairs (MAT-10,MAT-4), (MAT-4,VITS), (VITS,MAT-2), (MAT-2,GCFM-4), and (GCFM-4,GRAD-10). This means that MAT always had significantly better rated naturalness than GRAD for the same NFE, and always surpassed FS2. Both the new architecture and training method contributed to the naturalness improvement, since MAT-4>GCFM-4>GRAD-4. The fact that GRAD-10 was much better than GRAD-4 whilst MAT-10 and MAT-4 performed similarly suggests that GRAD requires many steps for good synthesis quality, whereas MAT reached a good level in fewer steps. Finally, VITS performed similarly to MAT-2 and MAT-4 in terms of MOS. MAT-10, although close to MAT-4 in rating, was significantly better than VITS. For any given n, MAT-n always scored higher than any system with equal or faster RTF. In summary, Matcha-TTS achieved similar or better naturalness than all comparable baselines.

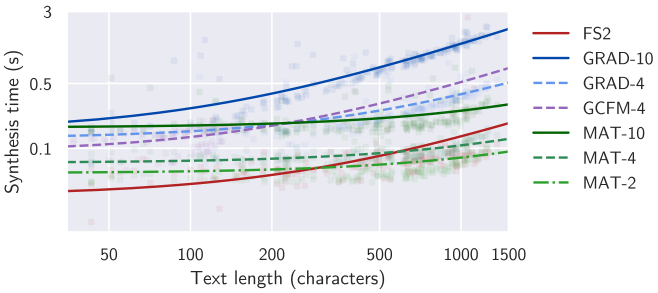

Fig. 3: Scatterplot of prompt length vs. synthesis time for acoustic models. Regression lines show as curves due to the log-log axes.

Finally, we evaluated how synthesis speed scaled with utterance length for the different models, by generating 180 sentences of different lengths using a GPT-2[8] model and plotting wall-clock synthesis time in Fig. 3, also fitting least-squares regression lines to the data. The results show that MAT-2 synthesis speed becomes competitive with FS2 at longer utterances, with MAT-4 not far behind. The major contributor to this appears to be the new architecture (since GRAD-4 and GCFM-4 both are much slower), and the gap from MAT to GRAD only grows with longer utterances.

5. CONCLUSIONS AND FUTURE WORK

We have introduced Matcha-TTS, a fast, probabilistic, and high-quality ODE-based TTS acoustic model trained using conditional flow matching. The approach is non-autoregressive, memory efficient, and jointly learns to speak and align. Compared to three strong pre-trained baselines, Matcha-TTS provides superior speech naturalness and can match the speed of the fastest model on long utterances. Our experiments show that both the new architecture and the new training contribute to these improvements.

Compelling future work includes making the model multi-speaker, adding probabilistic duration modelling, and applications to challenging, diverse data such as spontaneous speech [42].

6. REFERENCES

[1] Y. Song and S. Ermon, "Generative modeling by estimating gradients of the data distribution," *Proc. NeurIPS*, 2019.

[2] P. Dhariwal and A. Nichol, "Diffusion models beat GANs on image synthesis," in *Proc. NeurIPS*, 2021, pp. 8780–8794.

[8] https://huggingface.co/gpt2

[3] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. CVPR*, 2022, pp. 10 684–10 695.

[4] S. Alexanderson, R. Nagy, J. Beskow, and G. E. Henter, "Listen, denoise, action! Audio-driven motion synthesis with diffusion models," *ACM ToG*, vol. 42, no. 4, 2023, article 44.

[5] S. Mehta, S. Wang, S. Alexanderson, J. Beskow, É. Székely, and G. E. Henter, "Diff-TTSG: Denoising probabilistic integrated speech and gesture synthesis," in *Proc. SSW*, 2023.

[6] N. Chen, Y. Zhang, H. Zen, R. J. Weiss, M. Norouzi, and W. Chan, "WaveGrad: Estimating gradients for waveform generation," in *Proc. ICLR*, 2021.

[7] N. Chen, Y. Zhang, H. Zen, R. J. Weiss, M. Norouzi, N. Dehak, and W. Chan, "WaveGrad 2: Iterative refinement for text-to-speech synthesis," in *Proc. Interspeech*, 2021, pp. 3765–3769.

[8] V. Popov, I. Vovk, V. Gogoryan, T. Sadekova, and M. Kudinov, "Grad-TTS: A diffusion probabilistic model for text-to-speech," in *Proc. ICML*, 2021, pp. 8599–8608.

[9] M. Jeong, H. Kim, S. J. Cheon, B. J. Choi, and N. S. Kim, "Diff-TTS: A denoising diffusion model for text-to-speech," in *Proc. Interspeech*, 2021, pp. 3605–3609.

[10] Z. Kong, W. Ping, J. Huang, K. Zhao, and B. Catanzaro, "DiffWave: A versatile diffusion model for audio synthesis," in *Proc. ICLR*, 2021.

[11] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," in *Proc. ICLR*, 2021.

[12] M. S. Albergo and E. Vanden-Eijnden, "Building normalizing flows with stochastic interpolants," in *Proc. ICLR*, 2022.

[13] R. T. Q. Chen, Y. Rubanova, J. Bettencourt *et al.*, "Neural ordinary differential equations," in *Proc. NeurIPS*, 2018.

[14] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu *et al.*, "Flow matching for generative modeling," in *Proc. ICLR*, 2023.

[15] I. Vovk, T. Sadekova, V. Gogoryan, V. Popov, M. Kudinov, and J. Wei, "Fast Grad-TTS: Towards efficient diffusion-based speech generation on CPU," in *Proc. Interspeech*, 2022.

[16] J. Betker, "Better speech synthesis through scaling," *arXiv preprint arXiv:2305.07243*, 2023.

[17] Y. Ren, Y. Ruan, X. Tan, T. Qin, S. Zhao, Z. Zhao, and T.-Y. Liu, "FastSpeech: Fast, robust and controllable text to speech," in *Proc. NeurIPS*, 2019.

[18] Y. Ren, C. Hu, X. Tan, T. Qin, S. Zhao, Z. Zhao, and T.-Y. Liu, "FastSpeech 2: Fast and high-quality end-to-end text to speech," in *Proc. ICLR*, 2021.

[19] O. Press, N. A. Smith, and M. Lewis, "Train short, test long: Attention with linear biases enables input length extrapolation," in *Proc. ICLR*, 2022.

[20] J. Kim, S. Kim, J. Kong, and S. Yoon, "Glow-TTS: A generative flow for text-to-speech via monotonic alignment search," in *Proc. NeurIPS*, 2020, pp. 8067–8077.

[21] J. Kim, J. Kong, and J. Son, "VITS: Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech," in *Proc. ICML*, 2021, pp. 5530–5540.

[22] P. Shaw, J. Uszkoreit, and A. Vaswani, "Self-attention with relative position representations," in *Proc. NAACL*, 2018.

[23] H. Zhang, Z. Huang, Z. Shang, P. Zhang, and Y. Yan, "LinearSpeech: Parallel text-to-speech with linear complexity," in *Proc. Interspeech*, 2021, pp. 4129–4133.

[24] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced Transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021.

[25] U. Wennberg and G. E. Henter, "The case for translation-invariant self-attention in Transformer-based language models," in *Proc. ACL-IJCNLP Vol. 2*, 2021, pp. 130–140.

[26] S. Mehta, A. Kirkland, H. Lameris, J. Beskow, É. Székely, and G. E. Henter, "OverFlow: Putting flows on top of neural transducers for better TTS," in *Proc. Interspeech*, 2023.

[27] R. Huang, Z. Zhao, H. Liu, J. Liu, C. Cui, and Y. Ren, "ProDiff: Progressive fast diffusion model for high-quality text-to-speech," in *Proc. MM*, 2022, pp. 2595–2605.

[28] O. Watts, G. E. Henter, J. Fong, and C. Valentini-Botinhao, "Where do the improvements come from in sequence-to-sequence neural TTS?" in *Proc. SSW*, 2019, pp. 217–222.

[29] S. Mehta, É. Székely, J. Beskow, and G. E. Henter, "Neural HMMs are all you need (for high-quality attention-free TTS)," in *Proc. ICASSP*, 2022, pp. 7457–7461.

[30] S.-g. Lee, W. Ping, B. Ginsburg, B. Catanzaro, and S. Yoon, "BigVGAN: A universal neural vocoder with large-scale training," in *Proc. ICLR*, 2023.

[31] X. Liu *et al.*, "Flow straight and fast: Learning to generate and transfer data with rectified flow," in *Proc. ICLR*, 2023.

[32] Z. Ye, W. Xue, X. Tan, J. Chen, Q. Liu, and Y. Guo, "CoMoSpeech: One-step speech and singing voice synthesis via consistency model," in *Proc. ACM MM*, 2023, pp. 1831–1839.

[33] M. Le, A. Vyas, B. Shi, B. Karrer, L. Sari, R. Moritz *et al.*, "Voicebox: Text-guided multilingual universal speech generation at scale," *arXiv preprint arXiv:2306.15687*, 2023.

[34] M. Bernard and H. Titeux, "Phonemizer: Text to phones transcription for multiple languages in Python," *J. Open Source Softw.*, vol. 6, no. 68, p. 3958, 2021.

[35] J. Kong, J. Kim, and J. Bae, "HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis," in *Proc. NeurIPS*, 2020, pp. 17 022–17 033.

[36] R. Prenger, R. Valle, and B. Catanzaro, "WaveGlow: A flow-based generative network for speech synthesis," in *Proc. ICASSP*, 2019, pp. 3617–3621.

[37] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. Mcleavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. ICML*, 2023, pp. 28 492–28 518.

[38] J. Taylor and K. Richmond, "Confidence intervals for ASR-based TTS evaluation," in *Proc. Interspeech*, 2021.

[39] K. Prahallad, A. Vadapalli, N. Elluru, G. Mantena, B. Pulugundla *et al.*, "The Blizzard Challenge 2013 – Indian language task," in *Proc. Blizzard Challenge Workshop*, 2013.

[40] C.-H. Chiang, W.-P. Huang, and H. yi Lee, "Why we should report the details in subjective evaluation of TTS more rigorously," in *Proc. Interspeech*, 2023, pp. 5551–5555.

[41] A. Kirkland, S. Mehta, H. Lameris, G. E. Henter, E. Szekely *et al.*, "Stuck in the MOS pit: A critical analysis of MOS test methodology in TTS evaluation," in *Proc. SSW*, 2023.

[42] É. Székely, G. E. Henter, J. Beskow, and J. Gustafson, "Spontaneous conversational speech synthesis from found data," in *Proc. Interspeech*, 2019, pp. 4435–4439.